

Wordlists com Bash

■ Iniciante ■ Intermediário

■ GERAÇÃO BÁSICA

01 Wordlist com Expansão de Chaves

INIC.

Crie um script que gere uma wordlist usando **expansão de chaves** com uma palavra base passada como argumento (\$1) e um intervalo numérico de 1 a 100. Cada entrada deve ser gravada em uma linha separada no arquivo `wordlist.txt` usando `printf 's\n'`. Ao final, exiba o total de entradas geradas.

■ *Dica: use `printf 's\n' "${1}"{1..100} >> wordlist.txt` para gerar e salvar todas as combinações de uma vez.*

02 Gerador de Lista com Múltiplos Prefixos

INIC.

Escreva um script que declare um **array de prefixos** fixo no script (`admin`, `user`, `root`, `guest`, `test`) e, para cada prefixo, gere combinações com números de 1 a 50 usando `printf 's\n'` com expansão de chaves. Salve tudo em `wordlist.txt` e exiba o total ao final.

■ *Dica: dentro de um loop `for prefixo in "${array[@]}"` use `printf 's\n' "${prefixo}"{1..50}` para gerar cada bloco.*

■ VARIAÇÃO E MUTAÇÃO DE PALAVRAS

03 Gerador de Variações de Case

INIC.

Crie um script que leia um arquivo de palavras base (\$1) linha por linha e, para cada palavra, gere 4 variações usando **expansão de parâmetros**: original, `${var^^}` (tudo maiúsculo), `${var,,}` (tudo minúsculo) e `${var^}` (primeira maiúscula). Salve todas as variações em `variacoes.txt`.

■ *Dica: use `while read palavra; do ... done < "$1"` para ler o arquivo e gere as 4 variações dentro do loop.*

04 Mutador com Substituição de Caracteres (Leet Speak)

INTER.

Escreva um script que leia uma wordlist (\$1) e aplique **substituição de padrões** para gerar versões "leet speak" de cada palavra: `a→@`, `e→3`, `i→1`, `o→0`, `s→$`. Mantenha a palavra original e adicione a versão mutada logo abaixo. Salve em `leet.txt`.

■ *Dica: encadeie as substituições: `leet="${palavra//a/@}"; leet="${leet//e/3}"` e assim por diante para cada caractere.*

■ COMBINAÇÃO DE LISTAS

05

Produto Cartesiano de Duas Listas

INTER.

Crie um script que receba dois arquivos como argumentos (\$1 e \$2) e gere todas as combinações possíveis entre as palavras de cada arquivo usando loops `while` e `le` aninhados. Cada combinação deve ser uma concatenação direta das duas palavras, salva em `combinado.txt`. Exiba o total de combinações geradas.

■ *Dica: use dois loops aninhados com `while read p1` e `while read p2` e escreva `echo "${p1}${p2}"` dentro do mais interno.*

06

Gerador de Combinações com Separadores

INTER.

Escreva um script que declare dois arrays fixos: **palavras** (senha, pass, key) e **sufixos** (123, 2024, !, @#). Gere todas as combinações com e sem separador `_` entre eles usando loops `for` aninhados. Remova duplicatas com `sort -u` ao final.

■ *Dica: dentro dos loops aninhados, use `echo "${p}${s}"` e `echo "${p}_${s}"` para gerar as duas variantes de cada combinação.*

■ LIMPEZA E PÓS-PROCESSAMENTO

07

Removedor de Duplicatas e Ordenador

INIC.

Crie um script que receba uma wordlist como argumento (\$1) e realize o pós-processamento completo: remova linhas duplicadas com `sort -u`, remova linhas vazias com `grep -v '^$'` e remova palavras com menos de 4 caracteres usando um loop `while read`. Salve o resultado limpo em `wordlist_limpa.txt` e exiba quantas entradas foram removidas.

■ *Dica: calcule a diferença com `$(( antes - depois ))` usando `wc -l` antes e depois do processamento.*

08

Filtro por Comprimento e Complexidade

INTER.

Escreva um script que leia uma wordlist (\$1) e filtre apenas as palavras que atendam a **todos** os critérios: comprimento entre 8 e 20 caracteres (`${#palavra}`), conter ao menos um número (`[[ $palavra =~ [0-9] ]]`) e conter ao menos uma letra maiúscula. Salve as aprovadas em `complexas.txt`.

■ *Dica: use `[[ $palavra =~ [A-Z] ]]` para checar maiúsculas e `[[ ${#palavra} -ge 8 && ${#palavra} -le 20 ]]` para o comprimento.*

■ SCRIPT COMPLETO

09

Gerador de Wordlist Completo com Funções

INTER.

Crie um script completo de geração de wordlist com 4 funções: `gerar_numerica()` (palavra + números 1–100), `gerar_variacoes()` (maiúsculo, minúsculo, primeira maiúscula), `gerar_sufixos()` (adiciona !, @, 123, 2024 a cada palavra) e `limpar()` (remove duplicatas e ordena). Receba a palavra base via `$1`, chame todas as funções e exiba o total final.

■ *Dica: redirecione a saída de cada função com `>> "$arquivo"` e chame `limpar()` por último com `sort -u "$arquivo" -o "$arquivo"`.*

■ Dica para Iniciantes

Use sempre `printf '%s\n'` no lugar de `echo` para gravar wordlists — ele é mais rápido e garante uma entrada por linha. Para contar o total de palavras geradas:

```
$ wc -l wordlist.txt
```